

ClassyFM Public JSON API

The public, read-oriented JSON API for the ClassyFM mobile app. All endpoints are mounted under `/api/v1`.

- **Base URL:** `https://classyfm.co.id/api/v1`
 - **Format:** JSON (`Content-Type: application/json; charset=utf-8`)
 - **Orientation:** fully **read-only** and **cookie-free** — every endpoint is an open `GET`, with no authentication and no request body.
-

Table of Contents

- Conventions
 - Response envelopes
 - Errors
 - Caching
 - Pagination & filtering
 - URLs & images
 - Degraded mode
 - Index of Endpoints
 - App bootstrap endpoints
 - Content endpoints
 - Live endpoints
 - Playing the live stream in an app
 - When the backend API is unreachable
 - Advertising endpoints
 - Legal pages
 - Object reference
-

Conventions

Response envelopes

Shape	Used by	Body
List	roster/list endpoints	<code>{ "data": [...] }</code>
Paginated	<code>GET /news?source=...</code> , <code>GET /podcasts</code>	<code>{ "data": [...], "meta": { "page": 1, "total_pages": 3, "total": 27 } }</code>
Object	detail & aggregate endpoints	the object directly (no wrapper)

Errors

Errors return `{ "error": "message" }` with the relevant HTTP status:

Status	Meaning
400	Invalid path id or query parameter
404	Resource not found, or the feature is not configured
500	Server / database error

Caching

Each response carries a `Cache-Control` header:

- `public, max-age=60` — slowly-changing content (programs, news, podcasts, about, ads, home, config).
- `no-store` — live data (now-playing, schedule state, TikTok live).

Pagination & filtering

Pagination applies only to `GET /news?source=...` and `GET /podcasts`:

- `page` — 1-based, defaults to `1` (values `< 1` are treated as `1`).
- Page size is fixed at `12`.
- `meta.total_pages` is at least `1` even when empty.

URLs & images

All image and link fields are returned as **absolute URLs** (resolved against the site origin, `https://classyfm.co.id`). Already-absolute URLs — e.g. aggregated news thumbnails — pass through

unchanged.

Degraded mode

If the database is unavailable, **list** endpoints degrade gracefully to `{ "data": [] }` and **detail** endpoints return `404` — they never 500 for a missing DB.

Endpoints

App bootstrap endpoints

The two aggregate endpoints an app hits first at launch.

Method	Endpoint	Auth	Description
GET	/api/v1/config	open	App bootstrap: identity, stream, social links
GET	/api/v1/home	open	Aggregate home-screen feed in one request

Content endpoints

Method	Endpoint	Auth	Description
GET	/api/v1/programs	open	Active program roster, each flagged on-air
GET	/api/v1/programs/{slug}	open	One program with weekly schedule & broadcasters
GET	/api/v1/broadcasters	open	Active broadcaster roster
GET	/api/v1/broadcasters/{slug}	open	One broadcaster plus the programs they present
GET	/api/v1/news	open	Grouped news preview, or one source paginated
GET	/api/v1/news/{slug}	open	One <code>hot_release</code> article with gallery & related
GET	/api/v1/podcasts	open	Published podcasts, paginated
GET	/api/v1/podcasts/{slug}	open	One podcast with series & broadcasters
GET	/api/v1/podcast-series	open	Active podcast series list
GET	/api/v1/events	open	Published events & promos, paginated
GET	/api/v1/events/{slug}	open	One published event or promo
GET	/api/v1/about	open	About-page banner, segments & broadcaster preview

Live endpoints

Method	Endpoint	Auth	Description
GET	/api/v1/now-playing	open	Stream now-playing metadata & on-air program
GET	/api/v1/schedule/today	open	Today's schedule with live on-air/progress state
GET	/api/v1/schedule/current	open	The currently on-air program
GET	/api/v1/tiktok/live	open	TikTok live status

Advertising endpoints

Method	Endpoint	Auth	Description
GET	/api/v1/ads	open	Ad banners for a page, by placement slot

Legal pages

Method	Endpoint	Auth	Description
GET	/api/v1/legal	open	List of legal pages (slug, title, updated)
GET	/api/v1/legal/{slug}	open	One legal page: Markdown body + rendered HTML

App bootstrap endpoints

Both are `GET`, cached `public, max-age=60` — the endpoints an app calls at launch.

`GET /api/v1/config`

App bootstrap: station identity, stream URL, social links, and contact details.

```
{
  "station": { "name": "Classy 103.4 FM", "slogan": "..." },
  "stream_url": "https://c4.siar.us:10340/stream.mp3",
  "social": { "instagram": "https://...", "youtube": "https://..." },
  "contact": {
    "whatsapp_number": "0812...",
    "whatsapp_message": "Halo Classy FM...",
    "whatsapp_url": "https://wa.me/62812...?text=Halo%20Classy%20FM",
    "phone": "0751...",
    "phone_href": "tel:+62751...",
    "email": "info@classyfm.co.id"
  }
}
```

`social` keys are platform names as configured in the admin panel. `contact` is the admin-managed contact block (see `contact`); it is `null` if the backend has no database, and each of its fields is omitted when unset. The derived `whatsapp_url` (a ready `wa.me` deep link) and `phone_href` (`tel:`) are built server-side — prefer them over composing your own. The contact block returns whatever the admin saved, and the app decides how to surface it (e.g. a WhatsApp button).

`GET /api/v1/home`

Aggregate home-screen feed in a single request.

```
{
  "hero":    [ /* heroSlide objects */ ],
  "on_air":  { /* scheduleRow, or null */ },
  "programs": [ /* program objects, up to 6 */ ],
  "news":    [ /* newsGroup objects, up to 4 items each */ ]
}
```

See `heroSlide`.

Content endpoints

All are `GET` and cached `public, max-age=60`.

`GET /api/v1/programs`

Active program roster, each flagged with whether it is on air now.

```
{ "data": [  
  {  
    "title": "Morning Show",  
    "slug": "morning-show",  
    "description": "...",  
    "image_url": "https://classyfm.co.id/uploads/abc.jpg",  
    "url": "https://classyfm.co.id/program/morning-show",  
    "on_air": true  
  }  
] }
```

See `program` for the object fields.

`GET /api/v1/programs/{slug}`

One program with its full weekly schedule and broadcasters. `404` if the slug is unknown or the program is inactive.

```
{  
  "title": "Morning Show",  
  "slug": "morning-show",  
  "description": "...",  
  "image_url": "https://classyfm.co.id/uploads/abc.jpg",  
  "url": "https://classyfm.co.id/program/morning-show",  
  "on_air": true,  
  "schedule": [  
    { "from_day": 1, "to_day": 5, "start_time": "06:00", "end_time": "09:00", "host": "Anda, Yeni"  
  },  
  ],  
  "broadcasters": [ /* broadcaster objects */ ]  
}
```

`schedule[]` is a `scheduleGroup`; `broadcasters[]` are `broadcaster` objects.

GET /api/v1/broadcasters

Active broadcaster roster. `on_air` is `true` if any program the broadcaster presents is airing now.

```
{ "data": [ /* broadcaster objects */ ] }
```

GET /api/v1/broadcasters/{slug}

One broadcaster plus the programs they present. `404` if unknown.

```
{
  "name": "Anda",
  "slug": "anda",
  "role": "Announcer",
  "photo_url": "https://classyfm.co.id/uploads/anda.jpg",
  "bio": "...",
  "birth_place": "Padang",
  "birth_date": "...",
  "instagram": "...",
  "twitter": "...",
  "facebook": "...",
  "url": "https://classyfm.co.id/broadcasters/anda",
  "on_air": false,
  "programs": [ /* program objects */ ]
}
```

GET /api/v1/news

Two modes:

Without `source` (or with an unrecognized one) — grouped preview, up to 6 items per source group:

```
{ "data": [
  { "source": "klikpositif", "label": "KlikPositif", "items": [ /* news items */ ] },
  { "source": "katasumbar", "label": "KataSumbar", "items": [ ... ] },
  { "source": "hot_release", "label": "Hot Release", "items": [ ... ] },
  { "source": "youtube", "label": "YouTube", "items": [ ... ] }
] }
```

With a valid `source` — that source's items, paginated:

Query param	Notes
<code>source</code>	One of <code>youtube</code> , <code>klikpositif</code> , <code>katasumber</code> , <code>hot_release</code> . Invalid values fall back to grouped mode.
<code>page</code>	1-based, page size 12.

```
{ "data": [ /* news items */ ], "meta": { "page": 1, "total_pages": 4, "total": 42 } }
```

Items are `newsItem` objects. Note: `hot_release` items link to the on-site `/news/{slug}`; other (aggregated) sources keep their external `url`.

GET `/api/v1/news/{slug}`

One `hot_release` article, with split paragraphs, a mid-article image gallery, and related articles. `404` if unknown (only `hot_release` articles have on-site detail).

```
{
  "source": "hot_release",
  "source_label": "Hot Release",
  "title": "...",
  "slug": "...",
  "excerpt": "...",
  "image_url": "https://...",
  "url": "https://classyfm.co.id/news/...",
  "published_at": "2026-08-09T10:00:00Z",
  "is_featured": true,
  "paragraphs": ["...", "..."],
  "gallery_index": 3,
  "middle_images": ["https://...", "https://..."],
  "related": [ /* news items */ ]
}
```

`gallery_index` is the paragraph index after which the `middle_images` gallery should be inserted.

GET `/api/v1/podcasts`

Published podcasts, newest first, paginated. Only episodes whose series is **active** are returned — toggling a series inactive in the admin panel removes its episodes from both `data` and `meta.total`.

Query param	Notes
<code>series</code>	Optional series slug filter. An unknown/invalid slug is ignored (returns all); a slug that belongs to an <i>inactive</i> series returns an empty list.
<code>page</code>	1-based, page size 12.

```
{ "data": [ /* podcast objects */ ], "meta": { "page": 1, "total_pages": 2, "total": 18 } }
```

GET `/api/v1/podcasts/{slug}`

One podcast with its series name and broadcasters. `404` if unknown — this also covers a published podcast whose series has been marked inactive.

```
{
  "title": "...",
  "slug": "...",
  "description": "...",
  "spotify_url": "https://open.spotify.com/...",
  "thumb_url": "https://...",
  "series_name": "...",
  "url": "https://classyfm.co.id/podcast/...",
  "broadcasters": [ /* broadcaster objects */ ]
}
```

GET `/api/v1/podcast-series`

The **active** podcast series (for a filter picker feeding `?series=`). Series marked inactive in the admin panel are omitted.

```
{ "data": [ { "name": "...", "slug": "..." } ] }
```

GET `/api/v1/events`

Published events and promos, newest first, paginated.

Query param	Notes
<code>category</code>	Optional. One of <code>event</code> , <code>promo</code> . An unknown/invalid value is ignored (returns all).
<code>page</code>	1-based, page size 12.

```
{ "data": [ /* event objects */ ], "meta": { "page": 1, "total_pages": 1, "total": 5 } }
```

Items are `event` objects.

GET `/api/v1/events/{slug}`

One published event or promo. `404` if unknown or unpublished.

```
{
  "title": "...",
  "slug": "...",
  "category": "event",
  "description": "...",
  "image_url": "https://...",
  "event_date": "2026-09-01T00:00:00Z",
  "location": "...",
  "link_url": "https://...",
  "url": "https://classyfm.co.id/event/..."
}
```

`event_date`, `location`, `link_url`, `image_url`, and `description` are omitted when unset. `url` is the on-site event page.

GET `/api/v1/about`

About-page content: banner, text segments, and a broadcaster preview (max 8).

```
{
  "banner": {
    "media_type": "video",
    "image_url": "https://...",
    "video_url": "https://youtu.be/...",
    "embed_url": "https://www.youtube.com/embed/..."
  },
  "segments": [ { "segment": "profile", "title": "...", "body": "..." } ],
  "broadcasters": [ /* broadcaster objects */ ]
}
```

`banner.embed_url` is present only when `media_type` is `"video"` and the video URL is a resolvable YouTube link. `segment` is one of `profile`, `music`, `audience`.

Live endpoints

All are `GET`, cached `no-store` — live stream state and schedule progress.

`GET` `/api/v1/now-playing`

Stream now-playing metadata, plus the on-air program when the stream is live.

```
{
  "artist": "...",
  "song": "...",
  "has_song": true,
  "cover_url": "https://...",
  "live": true,
  "program": { /* scheduleRow, present only when live and a program is on air */ }
}
```

See `scheduleRow`. (The Shoutcast listener count is deliberately not exposed here — it is admin-only.)

`GET` `/api/v1/schedule/today`

Today's full schedule with live on-air/progress state.

```
{ "data": [ /* scheduleRow objects */ ] }
```

`GET` `/api/v1/schedule/current`

The currently on-air program as a single `scheduleRow` object, or `{ "on_air": false }` when nothing is airing.

`GET` `/api/v1/tiktok/live`

```
{ "live": true, "title": "..." }
```

Playing the live stream in an app

The audio stream is a **direct external Shoutcast MP3** — it lives on the Shoutcast host, **not** on `classyfm.co.id`, and this API never proxies or redirects the audio. The app plays it **directly**:

1. **Bootstrap the URL.** Read `stream_url` from `/api/v1/config` once at startup and hand it to the device's native audio player. It is a plain, unauthenticated streaming MP3 (Shoutcast) — no headers,

no token, no proxy. Prefer reading it from `config` over hardcoding it, so the Shoutcast host can be moved server-side without an app release.

2. Drive now-playing from a poll loop — via this API, never Shoutcast directly. Poll

`/api/v1/now-playing` on a timer to update the now-playing UI (and any lock-screen / notification metadata). The API fetches and caches the track metadata and `live` state from the Shoutcast server for you, so the app avoids CORS and doesn't hammer the Shoutcast box — **do not scrape the Shoutcast endpoints yourself**. The response is `no-store` but the server refreshes its upstream metadata only every ~12 s, so **polling faster than ~15 s gains nothing** — settle on roughly a 15 s interval while the player is active, and pause polling when it is stopped or backgrounded without audio.

- Show `artist` + `song` when `has_song` is `true`; when `false` there is no track metadata (station ID / no title) — fall back to the station name.
- Use `cover_url` for artwork, but it is best-effort and may be `""` — fall back to a bundled placeholder or the on-air program image.

3. Handle live vs. off-air. Switch the UI between "on air" and "off air" on the `live` flag. When `live` is `true`, the optional `program` object (a `scheduleRow`) gives the current show, host, and `progress` (0–100); `/api/v1/schedule/current` returns the same thing standalone, and `/api/v1/schedule/today` backs an "up next" list.

When the backend API is unreachable

Because the audio is a **direct Shoutcast MP3** and this API never proxies it, backend downtime (network error, timeout, `5xx`) does **not** interrupt playback — only the *metadata* around it. Keep the audio running and degrade only the now-playing chrome:

1. **Persist `config`**. Cache the last successful `/api/v1/config` response (at minimum `stream_url`, plus station name/slogan and social links) in local storage. At launch, start playback from the cached `stream_url` even when `config` can't be re-fetched. There is **no bundled/hardcoded stream URL** — the app never invents one. On a true cold start (first-ever launch, nothing cached, and the API unreachable) there is no URL to play: show a "stream unavailable / retry" state and fetch `config` again once connectivity returns.
2. **Keep audio alive when `now-playing` fails**. A failed/timed-out/`5xx` `/api/v1/now-playing` poll is a metadata gap, not a stream failure — never stop or reset the player because of it. Hold the last-known `artist` / `song` and `live` state, or fall back to the station name + bundled placeholder art.
3. **Back off, then recover**. On repeated poll failures, widen the interval (e.g. exponential backoff up to ~60 s) instead of hammering. On the next successful poll, resume the normal ~15 s cadence, refresh the now-playing UI, and re-cache `config`. No user action or app restart should be required.

A live listener count is intentionally not available to apps.

Advertising endpoints

GET, cached `public, max-age=60`.

GET `/api/v1/ads`

Ad banners for a page, grouped into `top` and `bottom` placement slots.

Query param	Notes
<code>page</code>	Target page key. Invalid keys return <code>400</code> . Omitted returns only banners targeted at every page.

Valid `page` keys: `home`, `about`, `program`, `program_detail`, `live`, `news`, `news_detail`, `broadcasters`, `broadcaster_detail`.

```
{
  "top": {
    "slideshow": false,
    "rotate_ms": 6000,
    "placeholder": false,
    "placeholder_text": "",
    "banners": [
      {
        "image_url": "https://classyfm.co.id/uploads/ad.jpg",
        "link_url": "https://sponsor.example",
        "alt": "...",
        "title": "..."
      }
    ]
  },
  "bottom": { /* same shape */ }
```

Each slot's `banners` is an array (empty when the slot has none). `slideshow` tells the client to rotate banners every `rotate_ms` rather than stack them; `placeholder` (with `placeholder_text`) says an empty slot should hold its space rather than collapse.

Legal pages

The **Privacy Policy** and **Terms & Conditions** are admin-managed content served as JSON, so an app can render them natively (or in a WebView from `body_html`) on a Settings/Legal screen. Both are `GET`, cached `public, max-age=60`. The pages are in Indonesian and their prose may change over time, so re-fetch rather than bundling a copy.

`GET` `/api/v1/legal`

The list of legal pages, enough to build a menu without shipping the full body.

```
{ "data": [
  { "slug": "privacy", "title": "Kebijakan Privasi", "updated_at": "2026-08-10T09:00:00Z" },
  { "slug": "terms", "title": "Syarat dan Ketentuan", "updated_at": "2026-08-10T09:00:00Z" }
] }
```

`GET` `/api/v1/legal/{slug}`

One legal page in full. `{slug}` is one of `privacy`, `terms`; any other value (or a missing page) returns `404`.

```
{
  "slug": "privacy",
  "title": "Kebijakan Privasi",
  "intro": "...",
  "body": "## Markdown source ...",
  "body_html": "<h2>...</h2>",
  "updated_at": "2026-08-10T09:00:00Z"
}
```

`body` is the Markdown source; `body_html` is the same content rendered to sanitized HTML server-side — render whichever your UI prefers. See `legalPage`.

The pages are also reachable as HTML at `https://classyfm.co.id/privacy-policy` and `https://classyfm.co.id/terms-and-conditions` (the website's own routes).

Object reference

Fields marked (*optional*) are omitted from the JSON when empty.

program

Field	Type	Notes
<code>title</code>	string	
<code>slug</code>	string	
<code>description</code>	string	(<i>optional</i>)
<code>image_url</code>	string	(<i>optional</i>) absolute URL
<code>url</code>	string	on-site program URL
<code>on_air</code>	bool	airing now

scheduleGroup

Field	Type	Notes
<code>from_day</code>	int	0 = Sunday ... 6 = Saturday
<code>to_day</code>	int	end of the day range (same numbering)
<code>start_time</code>	string	<code>HH:MM</code>
<code>end_time</code>	string	<code>HH:MM</code>
<code>host</code>	string	(<i>optional</i>) effective broadcaster set, e.g. <code>"Anda, Yeni"</code>

broadcaster

Field	Type	Notes
<code>name</code>	string	
<code>slug</code>	string	
<code>role</code>	string	<i>(optional)</i>
<code>photo_url</code>	string	<i>(optional)</i> absolute URL
<code>bio</code>	string	<i>(optional)</i>
<code>birth_place</code>	string	<i>(optional)</i>
<code>birth_date</code>	string	<i>(optional)</i>
<code>instagram</code>	string	<i>(optional)</i>
<code>twitter</code>	string	<i>(optional)</i>
<code>facebook</code>	string	<i>(optional)</i>
<code>url</code>	string	on-site broadcaster URL
<code>on_air</code>	bool	presenting an airing program now

newsItem

Field	Type	Notes
<code>source</code>	string	<code>youtube</code> <code>klikpositif</code> <code>katasumbar</code> <code>hot_release</code>
<code>source_label</code>	string	human-readable source name
<code>title</code>	string	
<code>slug</code>	string	<i>(optional)</i> present for <code>hot_release</code>
<code>excerpt</code>	string	<i>(optional)</i>
<code>image_url</code>	string	<i>(optional)</i>
<code>url</code>	string	on-site for <code>hot_release</code> , external otherwise
<code>published_at</code>	string	RFC 3339 timestamp
<code>is_featured</code>	bool	<i>(optional)</i>

newsGroup

Field	Type	Notes
<code>source</code>	string	source key
<code>label</code>	string	display label
<code>items</code>	newsItem[]	

podcast

Field	Type	Notes
<code>title</code>	string	
<code>slug</code>	string	
<code>description</code>	string	<i>(optional)</i>
<code>spotify_url</code>	string	<i>(optional)</i>
<code>thumb_url</code>	string	<i>(optional)</i> absolute URL
<code>series_name</code>	string	<i>(optional)</i>
<code>broadcaster</code>	string	<i>(optional)</i> list-view only
<code>url</code>	string	on-site podcast URL

event

Field	Type	Notes
<code>title</code>	string	
<code>slug</code>	string	
<code>category</code>	string	<code>event</code> or <code>promo</code>
<code>description</code>	string	<i>(optional)</i>
<code>image_url</code>	string	<i>(optional)</i> absolute URL
<code>event_date</code>	string	<i>(optional)</i> RFC 3339 timestamp
<code>location</code>	string	<i>(optional)</i>
<code>link_url</code>	string	<i>(optional)</i> external link
<code>url</code>	string	on-site event URL

scheduleRow

Field	Type	Notes
<code>title</code>	string	program title
<code>slug</code>	string	<i>(optional)</i>
<code>host</code>	string	<i>(optional)</i>
<code>image</code>	string	<i>(optional)</i> absolute URL
<code>start_time</code>	string	<code>HH:MM</code>
<code>end_time</code>	string	<code>HH:MM</code>
<code>on_air</code>	bool	
<code>progress</code>	int	0–100, % through the slot
<code>ended</code>	bool	slot already finished today

heroSlide

Field	Type	Notes
<code>href</code>	string	<i>(optional)</i> link target
<code>target_blank</code>	bool	<i>(optional)</i> open in new tab
<code>image_url</code>	string	<i>(optional)</i>
<code>badge_label</code>	string	<i>(optional)</i>
<code>title</code>	string	
<code>excerpt</code>	string	<i>(optional)</i>
<code>date</code>	string	<i>(optional)</i> RFC 3339 timestamp

contact

Returned inline under `contact` on `/config`. Optional fields are omitted when unset; the whole object is `null` if the backend has no database.

Field	Type	Notes
<code>whatsapp_number</code>	string	<i>(optional)</i> display form, as typed by the admin
<code>whatsapp_message</code>	string	<i>(optional)</i> prefilled message
<code>whatsapp_url</code>	string	<i>(optional)</i> ready <code>wa.me</code> deep link; omitted when no number
<code>phone</code>	string	<i>(optional)</i> display form
<code>phone_href</code>	string	<i>(optional)</i> <code>tel:++<digits></code> ; omitted when no phone
<code>email</code>	string	<i>(optional)</i>

legalPage

Field	Type	Notes
<code>slug</code>	string	<code>privacy</code> <code>terms</code>
<code>title</code>	string	
<code>intro</code>	string	<i>(optional)</i>
<code>body</code>	string	Markdown source
<code>body_html</code>	string	sanitized HTML rendered from <code>body</code>
<code>updated_at</code>	string	RFC 3339 timestamp